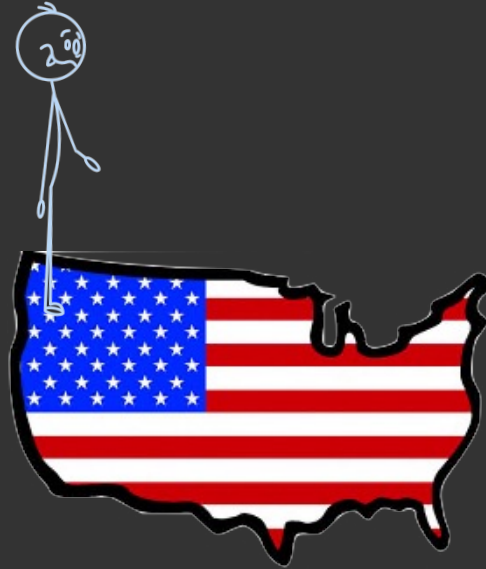


list comprehensions

CSCI
134

when i moved from the pacific northwest
to a private east coast college, people
told me i'd have to be more fashionable

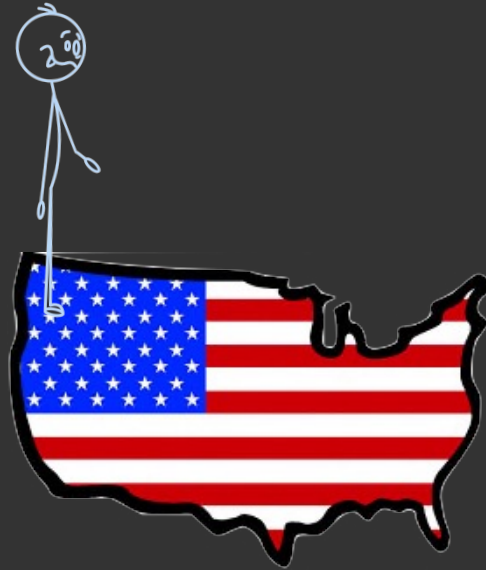
you'll have to be
more fashionable



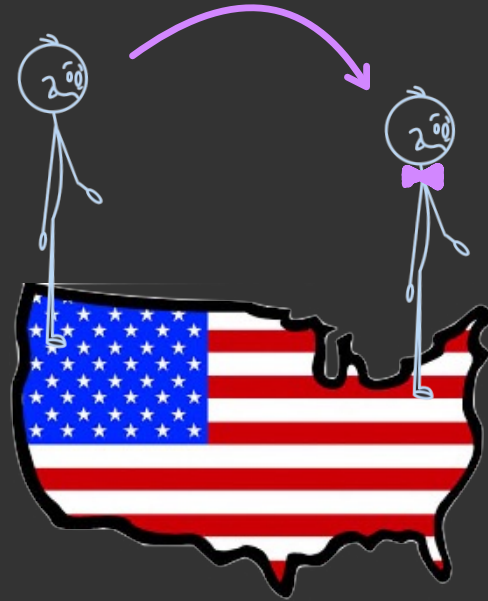
when i moved from the pacific northwest
to a private east coast college, people
told me i'd have to be more fashionable

you'll have to be
more fashionable

have you met
marque?



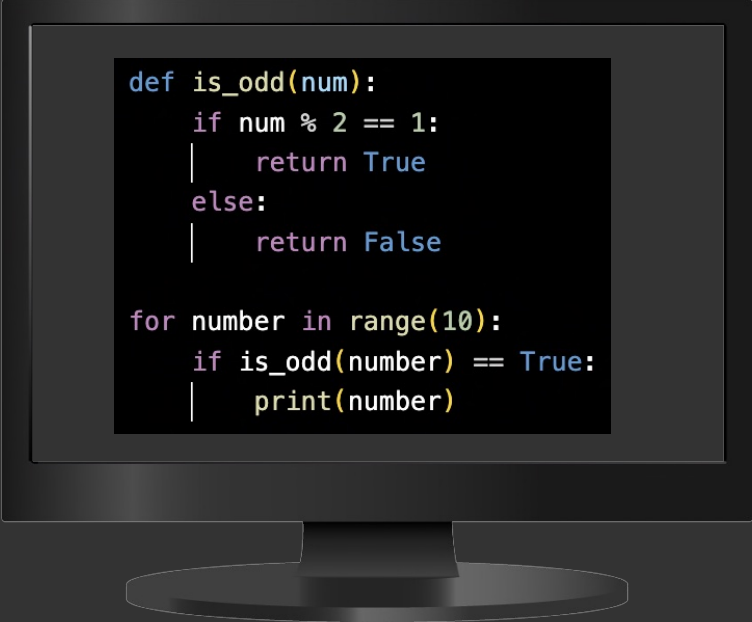
so i became more fashionable



likewise, there are ways to make
your python code more fashionable

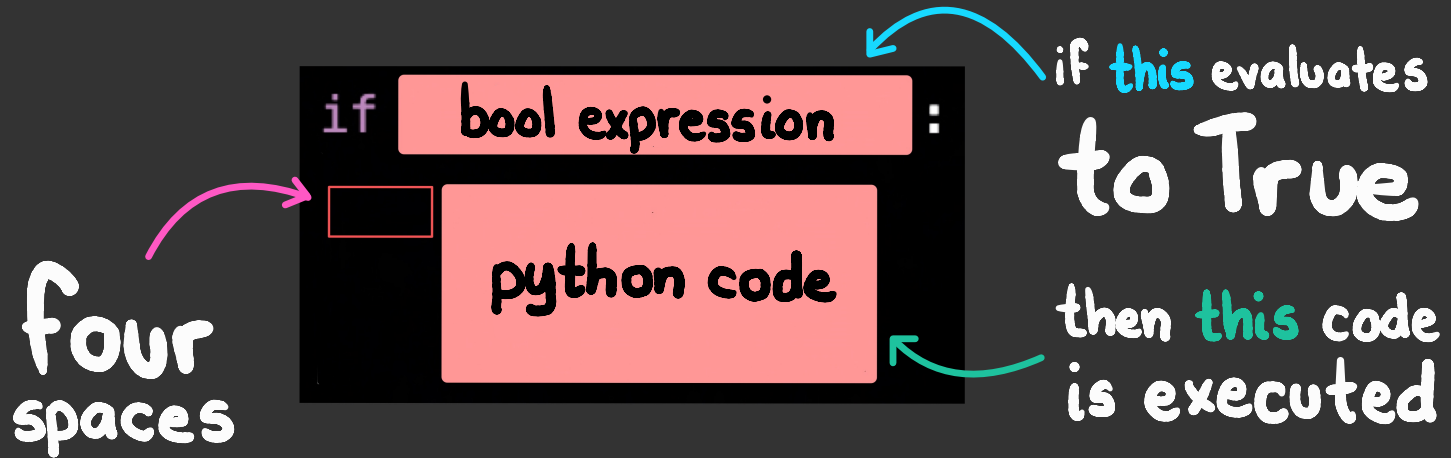


this code is a bit
unfashionable



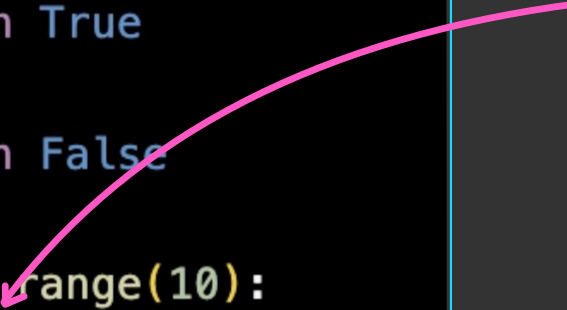
```
def is_odd(num):  
    if num % 2 == 1:  
        | return True  
    else:  
        | return False  
  
for number in range(10):  
    if is_odd(number) == True:  
        | print(number)
```

recall how the **if** statement works

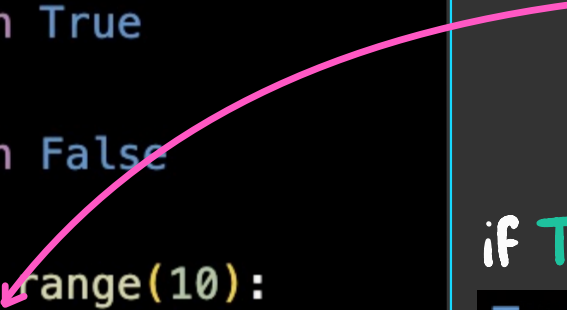


```
def is_odd(num):  
    if num % 2 == 1:  
        | return True  
    else:  
        | return False  
  
for number in range(10):  
    if is_odd(number) == True:  
        | print(number)
```

this is a **bool**
expression



```
def is_odd(num):  
    if num % 2 == 1:  
        | return True  
    else:  
        | return False  
  
for number in range(10):  
    if is_odd(number) == True:  
        | print(number)
```



this is a **bool**
expression

if **True**, then

True == True

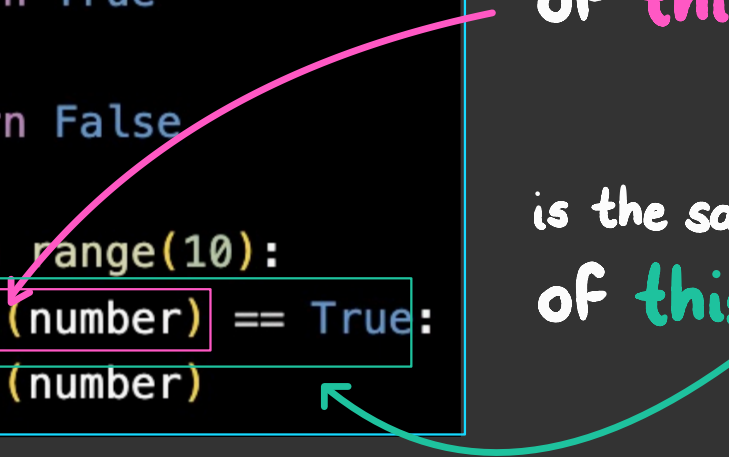
evaluates to **True**

if **False**, then

False == True

evaluates to **False**

```
def is_odd(num):  
    if num % 2 == 1:  
        | return True  
    else:  
        | return False  
  
for number in range(10):  
    if is_odd(number) == True:  
        | print(number)
```



either way, the value
of **this expression**

is the same as the value
of **this expression**

```
def is_odd(num):  
    if num % 2 == 1:  
        | return True  
    else:  
        | return False  
  
for number in range(10):  
    if is_odd(number):  
        | print(number)
```

so no need for

this



this is better, but still falls short of
the standards of python high fashion

```
def is_odd(num):  
    if num % 2 == 1:  
        | return True  
    else:  
        | return False  
  
for number in range(10):  
    if is_odd(number):  
        | print(number)
```



this is better, but still falls short of
the standards of python high fashion

```
def is_odd(num):  
    if num % 2 == 1:  
        return True  
    else:  
        return False  
  
for number in range(10):  
    if is_odd(number):  
        print(number)
```

if this evaluates to

True

then we return

True

this is better, but still falls short of
the standards of python high fashion

```
def is_odd(num):  
    if num % 2 == 1:  
        return True  
    else:  
        return False  
  
for number in range(10):  
    if is_odd(number):  
        print(number)
```

if this evaluates to

False

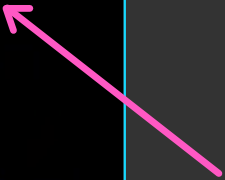
then we return

False

this is better, but still falls short of
the standards of **python high fashion**

```
def is_odd(num):  
    if num % 2 == 1:  
        | return True  
    else:  
        | return False  
  
for number in range(10):  
    if is_odd(number):  
        | print(number)
```

So we could
just return **this**
directly



```
def is_odd(num):  
    if num % 2 == 1:  
        | return True  
    else:  
        | return False  
  
for number in range(10):  
    if is_odd(number) == True:  
        | print(number)
```

unfashionable

```
def is_odd(num):  
    return num % 2 == 1  
  
for number in range(10):  
    if is_odd(number):  
        | print(number)
```

ooh la la

this code is perfectly fine,
but we can make it **pop**



```
woods = ["BALSA", "FIR", "BIRCH", "PINE"]
woodle_words = []
for wood in woods:
    if len(wood) == 5:
        |   woodle_words = woodle_words + [wood]
```



before

```
woods = ["BALSA", "FIR", "BIRCH", "PINE"]  
woodle_words = []  
for wood in woods:  
    if len(wood) == 5:  
        | woodle_words = woodle_words + [wood]
```



after

```
woods = ["BALSA", "FIR", "BIRCH", "PINE"]  
woodle_words = [wood for wood in woods if len(wood) == 5]
```

```
woodle_words = [wood for wood in woods if len(wood) == 5]
```

is equivalent to :

```
woodle_words = []  
for wood in woods:  
    if len(wood) == 5:  
        |   woodle_words = woodle_words + [wood]
```

```
accumulation variable = [ expression for variable in list if bool ]
```

is equivalent to :

```
accumulation variable = []  
for variable in list :  
    if bool :  
        | accumulation variable = accumulation variable + [ expression ]
```


this is called a
list comprehension

```
accumulation variable = [ expression for variable in list if bool ]
```

(it is considered very posh)

list comprehensions can be used to **filter** a list

```
In [1]: nums = [3, 1, 4, 1, 5, 9]
```

```
In [2]: [num for num in nums]
```

```
Out[2]: [3, 1, 4, 1, 5, 9]
```

the if
is optional!

```
In [3]: [num for num in nums if num > 3]
```

```
Out[3]: [4, 5, 9]
```

```
In [4]: [num for num in nums if num <= 3]
```

```
Out[4]: [3, 1, 1]
```

this is called **filtering**

list comprehensions can be used to **apply the same operation** to every value in a list

```
In [1]: nums = [3, 1, 4, 1, 5, 9]
```

```
In [2]: [num * 2 for num in nums]
```

```
Out[2]: [6, 2, 8, 2, 10, 18]
```

```
In [3]: [str(num) + "z" for num in nums]
```

```
Out[3]: ['3z', '1z', '4z', '1z', '5z', '9z']
```

```
In [4]: [num == 1 for num in nums]
```

```
Out[4]: [False, True, False, True, False, False]
```

this is called **mapping**

mapping and Filtering can be performed simultaneously

```
In [1]: nums = [3, 1, 4, 1, 5, 9]
```

```
In [2]: [str(num) + "z" for num in nums if num > 1]
```

```
Out[2]: ['3z', '4z', '5z', '9z']
```

```
In [3]: [num * 2 for num in nums if num > 1]
```

```
Out[3]: [6, 8, 10, 18]
```

```
In [4]: [num * 2 for num in nums if num <= 3]
```

```
Out[4]: [6, 2, 2]
```

write a list comprehension that
reverses every string in a list

```
In [1]: woods = ["ASPEN", "BALSA", "FIR", "CEDAR"]
```

```
In [2]: ?????  
Out[2]: ['NEPSA', 'ASLAB', 'RIF', 'RADEC']
```

```
accumulation variable = [ expression for variable in list if bool ]
```

write a list comprehension that reverses every string in a list

```
In [1]: woods = ["ASPEN", "BALSA", "FIR", "CEDAR"]
```

```
In [2]: [wood[::-1] for wood in woods]
```

```
Out[2]: ['NEPSA', 'ASLAB', 'RIF', 'RADEC']
```



```
accumulation variable = [ expression for variable in list if bool ]
```

write a list comprehension that keeps only the Floats in a list

```
In [1]: hodgepodge = ["ASPEN", 3, 3.14, "FIR", True, 2.0]
```

```
In [2]: ?????
```

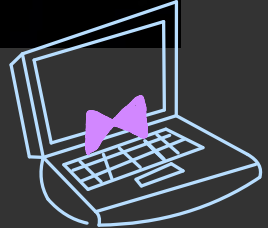
```
Out[2]: [3.14, 2.0]
```

```
accumulation variable = [ expression for variable in list if bool ]
```

write a list comprehension that keeps only the Floats in a list

```
In [1]: hodgepodge = ["ASPEN", 3, 3.14, "FIR", True, 2.0]
```

```
In [2]: [element for element in hodgepodge if type(element) == float]  
Out[2]: [3.14, 2.0]
```



```
accumulation variable = [ expression for variable in list if bool ]
```